

P2952R0

auto& operator=(X&&) = default

Published Proposal, 2023-08-10

Authors:

[Arthur O'Dwyer](#)

[Matthew Taylor](#)

Audience:

EWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Draft Revision:

12

Abstract

Current C++ forbids explicitly defaulted functions to have placeholder return types such as `auto&`, except for C++20's `operator<=>`. We remove this syntactic restriction in cases where the deduced return type would be the same as the expected one. This allows more consistency and less repetition when declaring defaulted functions.

Table of Contents

1 Changelog

2 Motivation

3 Proposal

- 3.1 "Return type" versus "declared return type"
- 3.2 "Defaulted as deleted"
- 3.3 "Deducing `this`" and CWG2586
- 3.4 Burden on specifying new defaultable operators
- 3.5 Existing corner cases
- 3.6 Impact on existing code

4 Implementation experience

5 Proposed wording

- 5.1 [class.eq]
- 5.2 [class.spaceship]
- 5.3 [class.compare.secondary]
- 5.4 [class.copy.assign]

References

Informative References

§ 1. Changelog

- R0:
 - Initial revision.

§ 2. Motivation

Current C++ permits `=default` to appear only on certain signatures, with certain return types. The current wording prohibits the use of placeholder types such as `auto&` to express these return types, with the single exception of C++20's `operator<=>`. This leads to redundant repetition, such as in this real code from libc++'s test suite:

```
struct ForwardDiffView {
    [...]
    ForwardDiffView(ForwardDiffView&&) = default;
    ForwardDiffView(const ForwardDiffView&) = default;
    ForwardDiffView& operator=(ForwardDiffView&&) = default;
    ForwardDiffView& operator=(const ForwardDiffView&) = default;
    [...]
};
```

We'd prefer to write the following, but the language doesn't currently permit it:

```
struct ForwardDiffView {
    [...]
    ForwardDiffView(ForwardDiffView&&) = default;
    ForwardDiffView(const ForwardDiffView&) = default;
    auto& operator=(ForwardDiffView&&) = default; // ill-formed C++23
    auto& operator=(const ForwardDiffView&) = default; // ill-formed C++23
    [...]
};
```

The comparison operators are inconsistent among themselves: `operator<=>` can deduce `strong_ordering`, but the others cannot deduce `bool`.

```
auto operator<=>(const MyClass& rhs) const = default; // well-formed
auto operator==(const MyClass& rhs) const = default; // ill-formed, must say 'bool'
auto operator<(const MyClass& rhs) const = default; // ill-formed, must say 'bool'
```

The status quo is inconsistent between non-defaulted and defaulted functions, making it unnecessarily tedious to upgrade to `=default`:

```
auto& operator=(const MyClass& rhs) { i = rhs.i; return *this; } // well-formed
auto& operator=(const MyClass& rhs) = default; // ill-formed, must say 'MyClass&'

auto operator==(const MyClass& rhs) const { return i == rhs.i; } // well-formed
auto operator==(const MyClass& rhs) const = default; // ill-formed, must say 'bool'
```

The ill-formedness of these declarations comes from overly restrictive wording in the standard, such as [\[class.eq\]/1](#) specifically requiring that a defaulted equality operator must have a *declared* return type of `bool`, instead of simply specifying that its return type must *be* `bool`. We believe each of the examples above has an intuitively clear meaning: the placeholder return type correctly matches the type which the defaulted body will actually return. We propose to loosen the current restrictions and permit these declarations to be well-formed.

This proposal does not seek to change the set of valid return types for these functions. We propose a purely syntactic change to expand the range of allowed declaration *syntax*, not semantics. (But we do one drive-by clarification which we believe matches EWG’s original intent: if an empty class’s defaulted `operator<=>` returns a non-comparison-category type, it should be defaulted as deleted.)

§ 3. Proposal

We propose that a defaulted function declaration with a placeholder return type should have its type deduced ([[dcl.spec.auto.general](#)]) as if from a fictional return statement that returns:

- a prvalue of type `bool`, in the case of `operator==`, `operator!=`, `operator<`, `operator>`, `operator<=`, or `operator>=`
- a prvalue of type `Q`, in the case of `operator<=>`; where `Q` is the common comparison type of $R_0, R_1, \dots, R_{n-1}$
- an lvalue of type `C`, in the case of `operator=` for a class or union type `C`

Then, the deduced return type is compared to the return type(s) permitted by the standard. If the types match, the declaration is well-formed. Otherwise it’s ill-formed.

EXAMPLE 1

For the copy-assignment operator, our proposal gives the following behavior:

```
struct MyClass {
    auto& operator=(const MyClass&) = default;           // Proposed OK: deduces MyClass&
    decltype(auto) operator=(const MyClass&) = default; // Proposed OK: deduces MyClass&
    auto&& operator=(const MyClass&) = default;          // Proposed OK: deduces MyClass&
    const auto& operator=(const MyClass&) = default;    // Still ill-formed: deduces const
    auto operator=(const MyClass&) = default;          // Still ill-formed: deduces MyCla
    auto* operator=(const MyClass&) = default;         // Still ill-formed: deduction fai
};
```

For `operator==`, our proposal gives the following behavior:

```
struct MyClass {
    auto operator==(const MyClass&) const = default;    // Proposed OK: deduces boo
    decltype(auto) operator==(const MyClass&) const = default; // Proposed OK: deduces boo
    auto&& operator==(const MyClass&) const = default;    // Still ill-formed: deduce
    auto& operator==(const MyClass&) const = default;    // Still ill-formed: deduct
};
```

§ 3.1. "Return type" versus "declared return type"

Today, vendors unanimously reject `auto& operator=(const A&) = default`. But we can’t find any wording in [[class.copy.assign](#)] or [[dcl.fct.def.default](#)] that directly justifies this behavior. It seems that vendors are interpreting e.g. [[dcl.fct.def.default](#)]/2.5’s “[if] the return type of F_1 differs from the return type of F_2 ” to mean “the *declared* return type of F_1 ,” even though newer sections such as [[class.compare](#)] consistently distinguish the “declared return type” from the (actual) return type.

We tentatively propose to leave [[dcl.fct.def.default](#)] alone, and simply add an example that indicates the (new) intent of the (existing) wording: that it should now be interpreted as talking about the assignment operator’s *actual* return type, not its *declared* (placeholder) return type.

§ 3.2. "Defaulted as deleted"

The current wording for comparison operators is crafted so that the following `Container` is well-formed. Its `operator<=>` is defaulted as deleted (so that operator is unusable), but the instantiation of class `Container<mutex>` itself is OK. We need to preserve this in our rewriting. (Godbolt.)

```
template<class T>
struct Container {
    T t;
    auto operator<=>(const Container&) const = default;
};

Container<std::mutex> cm;
// OK, <=> is deleted

struct Weird { int operator<=>(Weird) const; };
Container<Weird> cw;
// OK, <=> is deleted because Weird's operator<=>
// returns a non-comparison-category type
```

Similarly for dependent return types:

```
template<class R>
struct C {
    int i;
    R operator<=>(const C&) const = default;
};

static_assert(std::three_way_comparable<C<std::strong_ordering>>);
static_assert(!std::three_way_comparable<C<int>>);
// OK, C<int>'s operator<=> is deleted
```

Therefore we can't just say "`operator<=>` shall have a return type which is a comparison category type"; we must say that if the return type is not a comparison category type then the operator is defaulted as deleted.

§ 3.3. "Deducing `this`" and CWG2586

The resolution of [CWG2586] (adopted for C++23) permits defaulted functions to have explicit object parameters. This constrains the wordings we can choose for `operator=`: we can't say "the return type is deduced as if from `return *this`" because there might not be a `*this`.

There's a quirk with rvalue-ref-qualified assignment operators — not *move* assignment, but assignment where the *destination* object is explicitly rvalue-ref-qualified.

- `auto&& operator=(const B&) && { return *this; }` deduces an lvalue reference; but
- `auto&& operator=(this B&& self, const B&) { return self; }` deduces an rvalue reference.

Nonetheless, a *defaulted* assignment operator always returns an lvalue reference ([class.copy.assign]/6, [dcl.fct.def.default]/2.5), regardless of whether it's declared using explicit object syntax.

```
struct A {
    A& operator=(const A&) && = default; // OK today
    A&& operator=(const A&) && = default; // Ill-formed, return type isn't A&
    decltype(auto) operator=(const A&) && { return *this; } // OK, deduces A&
    decltype(auto) operator=(const A&) && = default; // Proposed OK, deduces A&
};
```

```

struct B {
    B& operator=(this B&& self, const B&) { return self; } // Error, self can't bind to B&
    B&& operator=(this B&& self, const B&) { return self; } // OK
    decltype(auto) operator=(this B&& self, const B&) { return self; } // OK, deduces B&&
    B& operator=(this B& self, const B&) = default; // OK
    B& operator=(this B&& self, const B&) = default; // OK
    B&& operator=(this B&& self, const B&) = default; // Ill-formed, return type isn't B&
    decltype(auto) operator=(this B&& self, const B&) = default; // Proposed OK, deduces B&
};

```

Defaulted rvalue-ref-qualified assignment operators are weird; Arthur is bringing another paper to forbid them entirely ([P2953]). However, P2952 doesn't need to treat them specially. Defaulted assignment operators invariably return lvalue references, so we invariably deduce as-if-from an lvalue reference, full stop.

§ 3.4. Burden on specifying new defaultable operators

We propose to leave `[dcl.fct.def.default]` alone and reinterpret its term "return type" to mean the actual return type, not the declared return type. This will, by default, permit the programmer to use placeholder return types on their defaulted operators. So there is a new burden on the specification of the defaultable operator, to specify exactly how return type deduction works for the implicitly defined operator.

EXAMPLE 2

[P1046] proposed to make `operator++(int)` defaultable in the same way as a secondary comparison operator. It would presumably have done this by adding wording to `[over.inc]`. After P2952, this added wording would need to include a sentence like:

A defaulted postfix `operator++` for class `X` shall have a return type that is `X` or `void`. If its declared return type contains a placeholder type, its return type is deduced as if from

- `return X(r);` where `r` is an lvalue reference to the function's object parameter, if `X(r)` is a well-formed expression;
- `return;` otherwise.

[P0847] §5.2's example of `add_postfix_increment` ("CRTP without the C, R, or even T") involves an `operator++` with return type `auto`; but that `operator++` is not defaulted, and probably couldn't be defaulted even after [P1046], firstly because it is a template and secondly because its deduced return type is `Self` instead of `add_postfix_increment`.

```

struct add_postfix_increment {
    template<class Self>
    auto operator++(this Self&, int) = default;
    // Today: ill-formed, can't default operator++
    // After P1046: presumably still not OK, can't default a template
};

struct S : add_postfix_increment {
    int i;
    auto& operator++() { ++i; return *this; }
    using add_postfix_increment::operator++;
};

S s = {1};
S t = s++;

```

§ 3.5. Existing corner cases

There is vendor divergence in some corner cases. Here is a table of the divergences we found, plus our opinion as to the conforming behavior, and our proposed behavior.

URL	Code	Clang	GCC	MSVC	EDG	
link	<code>const bool operator==(const C&) const = default;</code>	✗	✗	✗	✓	link
link	<code>friend bool operator==(const C, const C) = default;</code>	✓	✗	✓	✓	link
link	<code>decltype(auto) operator<=>(const C&) const = default;</code>	✗	✓	✗	✗	link
link	<code>const auto operator<=>(const C&) const = default;</code>	✗	✗	✓	✓	link
link	<code>True auto operator<=>(const C&) const = default;</code>	✗	✓	✓	✓	link
link	<code>False auto operator<=>(const C&) const = default;</code>	✗	unmet	✓	✓	link
link	<code>struct U { U(std::strong_ordering); }; struct C { U operator<=>(const C&) const = default; };</code>	✓	✓	✗	deleted	link
link	<code>struct U { U(std::strong_ordering); operator int(); }; struct C { int i; U operator<=>(const C&) const = default; };</code>	✓	✓	✗	deleted	link
link	<code>struct C { int i; const std::strong_ordering& operator<=>(const C&) const = default; };</code>	ICE	deleted	✓	deleted	link
link	<code>struct C { const std::strong_ordering& operator<=>(const C&) const = default; };</code>	ICE	✓	✓	deleted	link
link	<code>struct W { const std::strong_ordering operator<=>(const W&) const; }; struct C { W w; auto operator<=>(const C&) const = default; };</code>	✓	✗	✓	deleted	link
link	<code>struct W { const std::strong_ordering</code>	✓	✓	✓	✓	link

	<pre> operator<=>(const W&) const; }; struct C { W w; std::strong_ordering operator<=>(const C&) const = default; }; </pre>					
link	<pre> auto operator<=>(const M&) const noexcept(false) = default; </pre>	✓	✓	noexcept	inconsistent	✓
link	<pre> C& operator=(C&) = default; </pre>	✓	✓	✓	✓	✓
link	<pre> C& operator=(const C&&) = default; </pre>	deleted	X	X	deleted	d
link	<pre> C& operator=(const C&) const = default; </pre>	deleted	X	X	deleted	d
link	<pre> C& operator=(const C&) && = default; </pre>	✓	✓	✓	✓	✓
link	<pre> C&& operator=(const C&) && = default; </pre>	X	X	X	X	X

§ 3.6. Impact on existing code

There should be little effect on existing code, since this proposal mainly allows syntax that was ill-formed before. As shown in § 3.5 Existing corner cases, we do propose to change some very arcane examples, e.g.

```

struct C {
    const std::strong_ordering&
    operator<=>(const C&) const = default;
    // Today: Well-formed, non-deleted
    // Tomorrow: Well-formed, deleted
};

```

§ 4. Implementation experience

None yet.

§ 5. Proposed wording

§ 5.1. [class.eq]

NOTE: The phrase "equality operator function" ([over.binary]) covers both `==` or `!=`. But `!=` is not covered by [class.eq]; it's covered by [class.compare.secondary] below.

Modify [class.eq] as follows:

1. A defaulted equality operator function (`over::binary`) shall have a declared return type `bool`.

2. A defaulted `==` operator function for a class `C` is defined as deleted unless, for each x_i in the expanded list of subobjects for an object `x` of type `C`, $x_i == x_i$ is usable (`[class.compare.default]`).

3. The return value `V` of a defaulted `==` operator function with parameters `x` and `y` is determined by comparing corresponding elements x_i and y_i in the expanded lists of subobjects for `x` and `y` (in increasing index order) until the first index i where $x_i == y_i$ yields a result value which, when contextually converted to `bool`, yields `false`. If no such index exists, `V` is `true`. Otherwise, `V` is `false`.

x. A defaulted `==` operator function shall have the return type `bool`. If its declared return type contains a placeholder type, its return type is deduced as if from `return true`.

4. [Example 1:

```
struct D {
    int i;
    friend bool operator==(const D& x, const D& y) = default;
    // OK, returns x.i == y.i
};
```

— end example]

§ 5.2. [class.spaceship]

NOTE: There are only three "comparison category types" in C++, and `strong_ordering::equal` is implicitly convertible to all three of them. The status quo already effectively forbids `<=>` to return a non-comparison-category type, since either `R` is deduced as a common comparison type (which is a comparison category type by definition), or else a synthesized three-way comparison of type `R` must exist (which means `R` must be a comparison category type), or else the sequence x_i must be empty (in which case there are no restrictions on `R` except that it be constructible from `strong_ordering::equal`). We strengthen the wording to directly mandate that the return type be a comparison category type, even in the empty case.

Modify [class.spaceship] as follows:

[...]

2. Let `R` be the declared return type of a defaulted three-way comparison operator function, and let x_i be the elements of the expanded list of subobjects for an object `x` of type `C`.

— (2.1) If `R` is `auto`, contains a placeholder type, then let $cv_i R_i$ be the type of the expression $x_i <=> x_i$. The operator function is defined as deleted if that expression is not usable or if R_i is not a comparison category type (`[cmp.categories.pre]`) for any i . The return type is deduced as if from `return Q(std::strong_ordering::equal)`, where `Q` is the common comparison type (see below) of $R_0, R_1, \dots, R_{n-1}$.

— (2.2) Otherwise, ~~`R` shall not contain a placeholder type. If~~ if the synthesized three-way comparison of type `R` between any objects x_i and x_i is not defined, the operator function is defined as deleted.

3. The return value `V` ~~of type `R`~~ of the defaulted three-way comparison operator function with parameters `x` and `y` ~~of the same type~~ is determined by comparing corresponding elements x_i and y_i in the expanded lists of subobjects for `x` and `y` (in increasing index order) until the first index i where the synthesized three-way comparison of type `R` between x_i and y_i yields a result value v_i where $v_i != \emptyset$, contextually converted to `bool`, yields `true`; `V` is ~~a copy of~~ v_i . If no such index exists, `V` is ~~`static_cast<R>(std::strong_ordering::equal)`~~.

x. A defaulted three-way comparison operator function which is not deleted shall have a return type which is a comparison category type (`[cmp.categories.pre]`).

4. The common comparison type `U` of a possibly-empty list of n comparison category types `T0, T1, ..., Tn-1` is defined as follows:

[...]

§ 5.3. [class.compare.secondary]

Modify [class.compare.secondary] as follows:

1. A *secondary comparison operator* is a relational operator ([expr.rel]) or the != operator.

~~A defaulted operator function ([over.binary]) for a secondary comparison operator @ shall have a declared return type bool.~~

2. ~~The~~ A defaulted secondary comparison operator function with parameters x and y is defined as deleted if

— (2.1) overload resolution ([over.match]), as applied to x @ y, does not result in a usable candidate, or

— (2.2) the candidate selected by overload resolution is not a rewritten candidate.

Otherwise, the operator function yields x @ y. The defaulted operator function is not considered as a candidate in the overload resolution for the @ operator.

x. A defaulted secondary comparison operator function shall have the return type bool. If its declared return type contains a placeholder type, its return type is deduced as if from return true;.

3. [Example 1:

```
struct HasNoLessThan { };

struct C {
    friend HasNoLessThan operator<=>(const C&, const C&);
    bool operator<(const C&) const = default; // OK, function is deleted
};
```

— end example]

§ 5.4. [class.copy.assign]

NOTE: [class.copy.assign]/6 already clearly states that "The implicitly-declared copy/move assignment operator for class X has the return type X&." But we need this new wording to ensure that an *explicitly-defaulted* copy/move assignment operator will deduce that same type. (If it didn't deduce that type, then the explicitly-defaulted operator would be deleted, as in example B below.)

NOTE: Arthur initially proposed that [class.copy.assign]/14 should say "...returns an lvalue reference to the object for which...", but Jens Maurer thought that wouldn't be an improvement from CWG's point of view.

Modify [class.copy.assign] as follows:

14. The implicitly-defined copy/move assignment operator for a class returns the object for which the assignment operator is invoked, that is, the object assigned to.

15. If a defaulted copy/move assignment operator's declared return type contains a placeholder type, its return type is deduced as if from return r;, where r is an lvalue reference to the object for which the assignment operator is invoked.

16. [Example:

```
struct A {
    decltype(auto) operator=(A&&) = default;
    // Return type is A&
};
struct B {
    auto operator=(B&&) = default;
    // error: Return type is B, which violates [dcl.fct.def.default]/2.5
};
```

—end example]

§ References

§ Informative References

[CWG2586]

Barry Revzin. *Explicit object parameter for assignment and comparison*. May–July 2022. URL: <https://cplusplus.github.io/CWG/issues/2586.html>

[P0847]

Sy Brand; et al. *Deducing this*. July 2021. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2021/p0847r7.html>

[P1046]

David Stone. *Automatically Generate More Operators*. January 2020. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1046r2.html>

[P2953]

Arthur O'Dwyer. *Forbid defaulting operator=(X&&) &&*. August 2023. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2953r0.html>